Lecture 6. Efficient Inference

Luneva Natalia

17.03.2025

List of Content

Key-Value Cache

Reducing repeated computations

Key-Value Cache

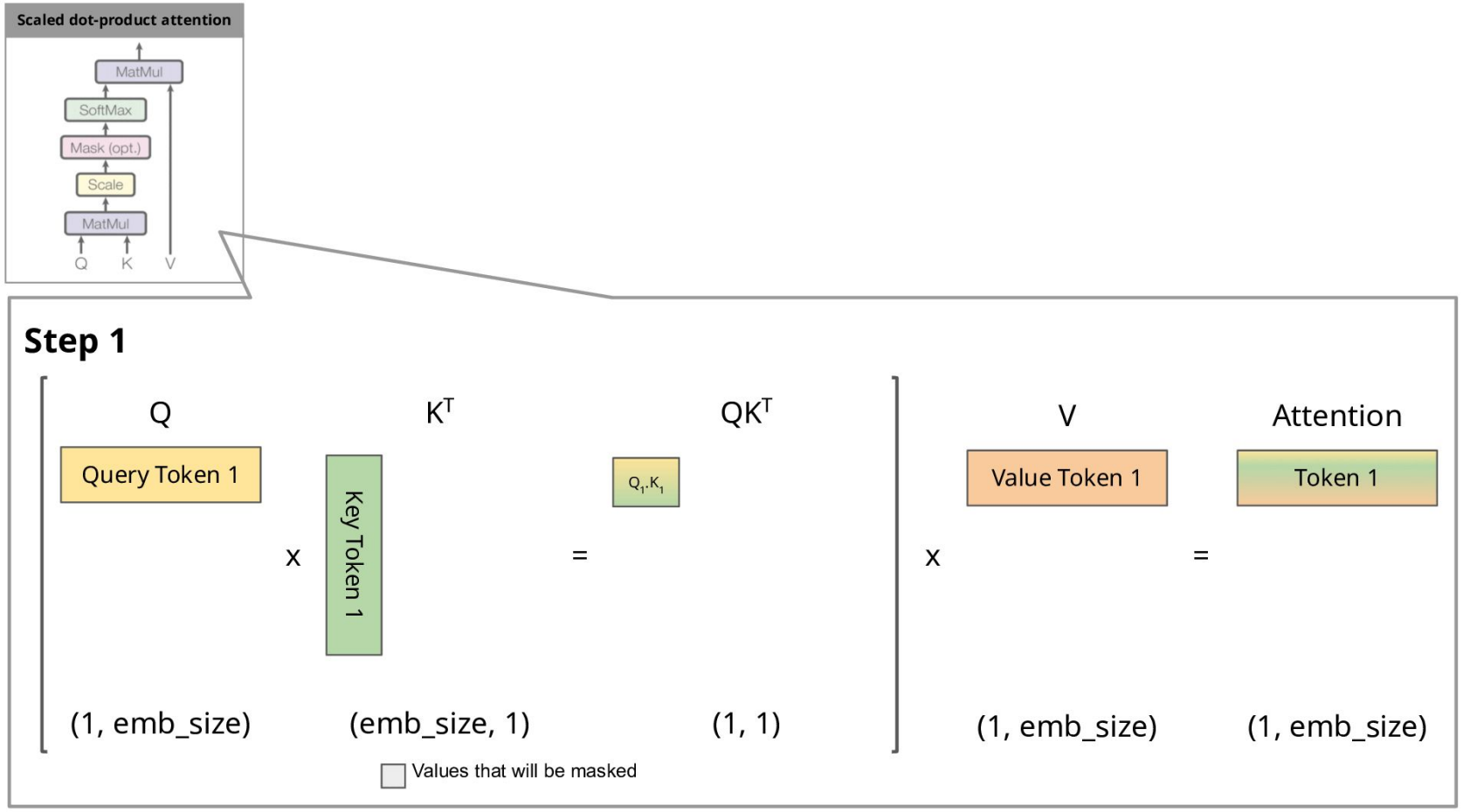
- Key-value vectors are used to calculate attention scores
- In autoregressive tasks (where the output at each step is generated based on the previous outputs) KV scores need to be recalculated, since the model predicts only one token at a time
- Repeating the same KV computation for every step is not efficient

Key-Value Cache

- Key-value vectors are used to calculate attention scores
- In autoregressive tasks (where the output at each step is generated based on the previous outputs) KV scores need to be recalculated, since the model predicts only one token at a time
- Repeating the same KV computation for every step is not efficient

KV-cache stores the past keys and values instead of recomputing them

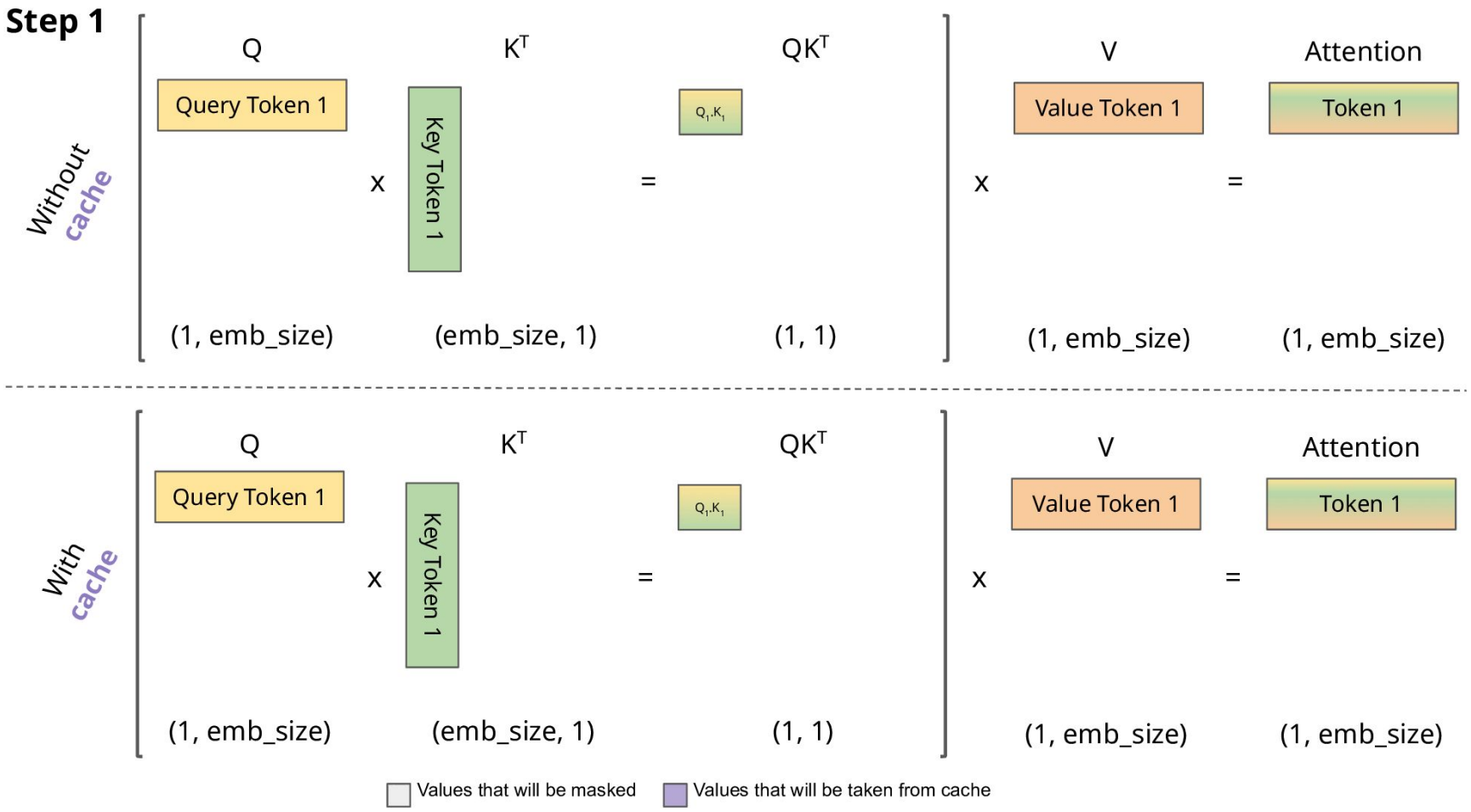
Key-Value Cache



Zoom-in! (simplified without Scale and Softmax)

[gif source]

Key-Value Cache



gif source

Key-Value Cache

Advantages:

- **Reduced Computation.** The same key and value matrices are reused across multiple decoding steps
- **Memory Reuse.** The model does not need to recompute and store intermediate results repeatedly
- **Scalability.** KV cache allows models to handle longer sequences more efficiently by avoiding the quadratic complexity associated with full attention computation

Key-Value Cache

Drawback:

- **Increased Memory Usage.** Caching the key and value matrices requires additional memory storage

KV Cache strategies

	Static	Dynamic
Definition	Keys and values are computed once and remain fixed	Keys and values are updated incrementally during decoding
Computation	Precomputes keys and values for the entire input sequence	Computes keys and values incrementally as new tokens are generated
Use Cases	• Encoder-decoder architectures (e.g. machine translation) • Batch inference in the autoregressive models with the common prefix (prompt)	Fully autoregressive models (e.g. text generation)

*There are also offloaded, quantized, and other types of cache

Paged Attention

Reducing memory wastes

Key Concepts

- **Contiguous Memory** is a type of memory allocation where data or processes are stored in a **single, continuous block** of memory addresses
- **Internal fragmentation.** A process is allocated more memory than it actually needs
- **External Fragmentation.** There are free memory blocks available in the system, but these blocks are too small to satisfy the memory requests of new processes

Memory Challenges in LLM Serving

- Existing LLM serving systems store the KV cache of a request **in contiguous memory space**
- The KV cache **adjusts its size dynamically** over time as the model generates new tokens
- Due to the unpredictable output lengths, serving systems statically **allocate a chunk of memory** for a request based on the request's **maximum possible sequence length**

Memory Challenges in LLM Serving

Memory wastes:

- **Reserved** slots for future tokens
- **Internal fragmentation** due to over-provisioning for potential maximum sequence lengths
- **External fragmentation** from the memory allocator like the buddy allocator

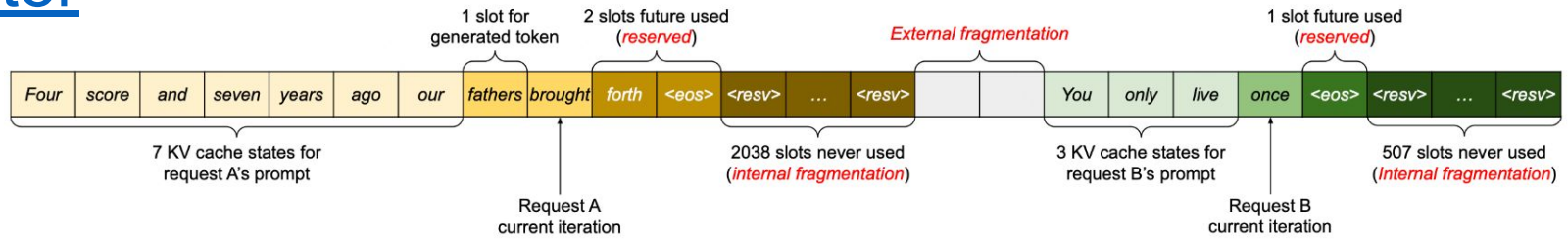


Figure 3. KV cache memory management in existing systems. Three types of memory wastes – reserved, internal fragmentation, and external fragmentation – exist that prevent other requests from fitting into the memory. The token in each memory slot represents its KV cache. Note the same tokens can have different KV cache when at different positions.

2309.06180

Memory Challenges in LLM Serving

- External fragmentation can also be significant, since the **pre-allocated size can be different for each request**
- Only **20% - 40%** of the KV cache memory was used to store the actual token states in the previous systems

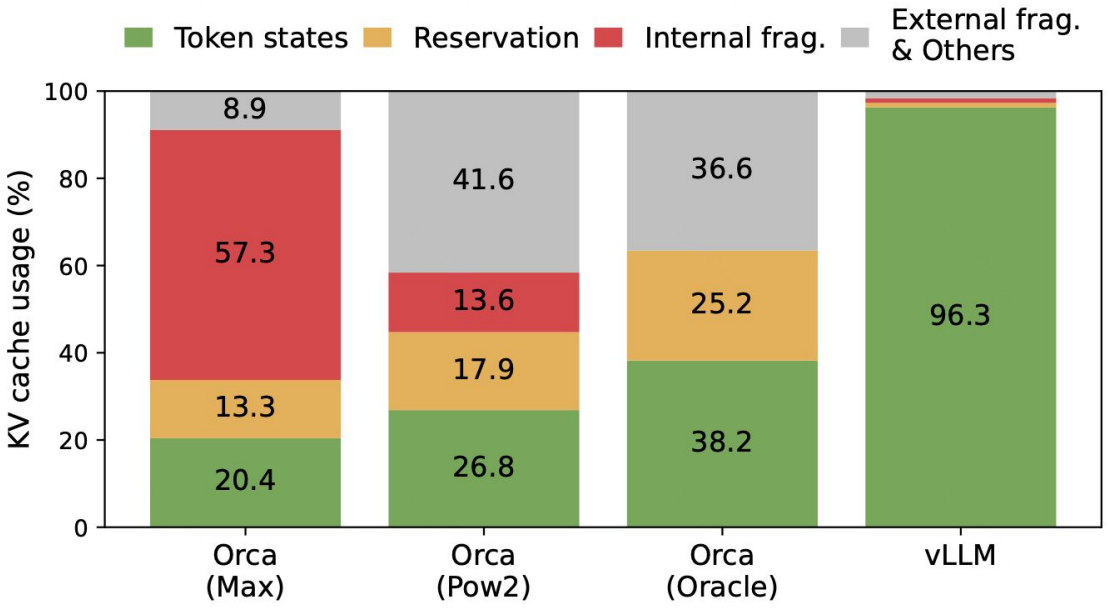


Figure 2. Average percentage of memory wastes in different LLM serving systems during the experiment in §6.2.

2309.06180

Paged Attention

- PagedAttention allocates **fixed-size** and relatively **small memory chunks** called blocks
- Each block can contain a fixed number of tokens and if necessary be **shared across different requests**

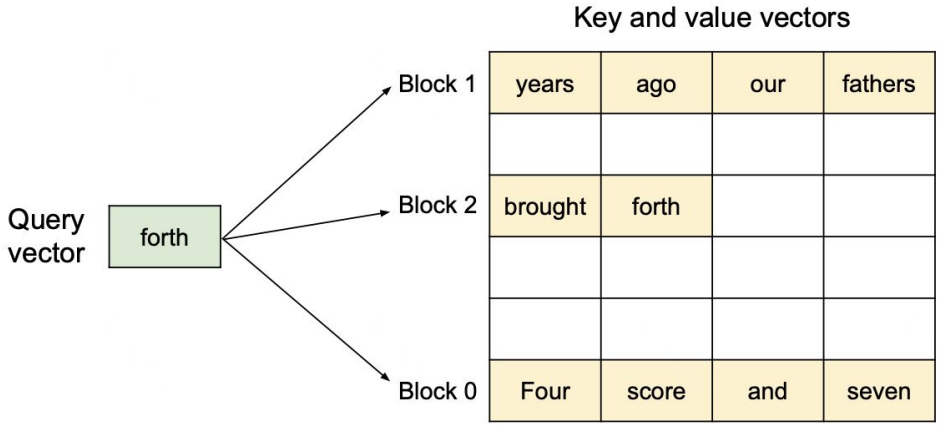


Figure 5. Illustration of the PagedAttention algorithm, where the attention key and values vectors are stored as non-contiguous blocks in the memory.

2309.06180

Paged Attention

- **On-demand allocation** and the **small block size** reduce internal memory fragmentation
- **Equally sized blocks** eliminate external memory fragmentation

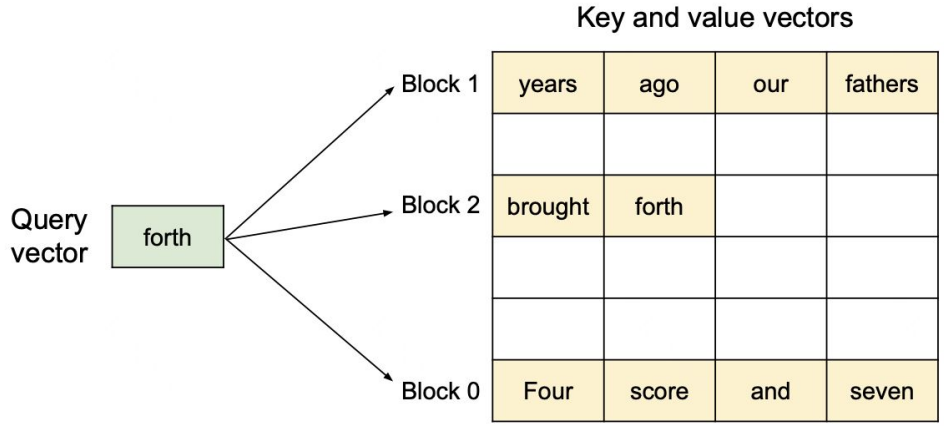
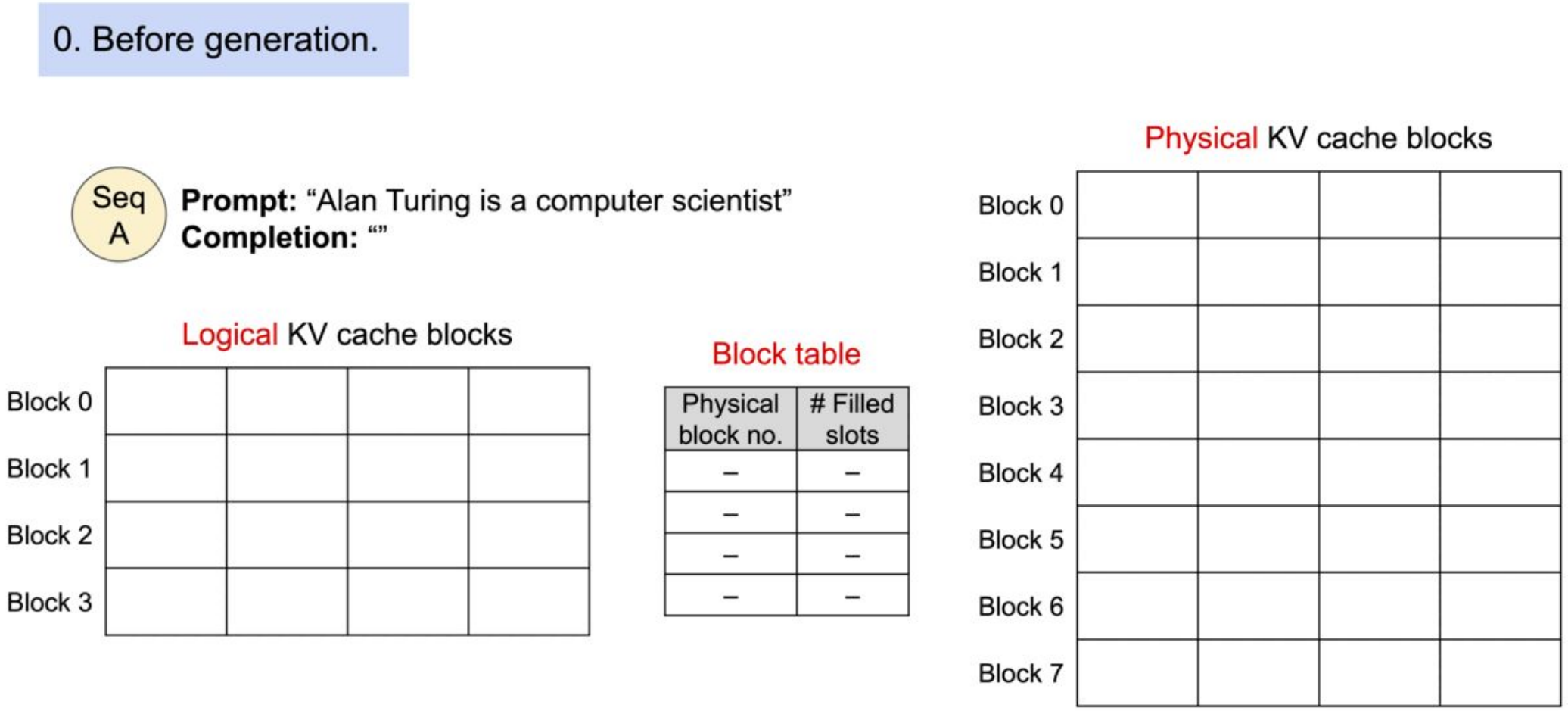


Figure 5. Illustration of the PagedAttention algorithm, where the attention key and values vectors are stored as non-contiguous blocks in the memory.

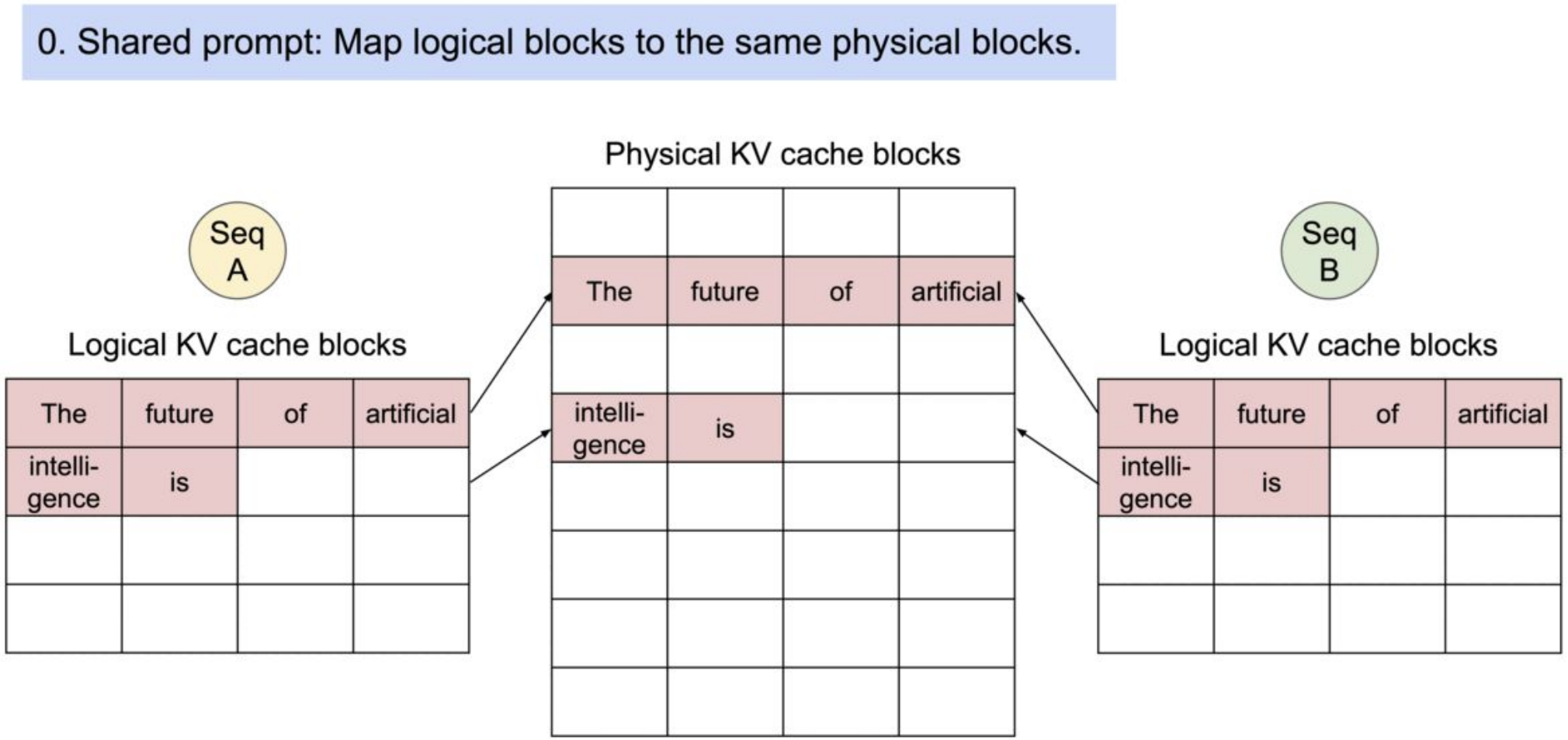
2309.06180

Paged Attention



gif source

Paged Attention



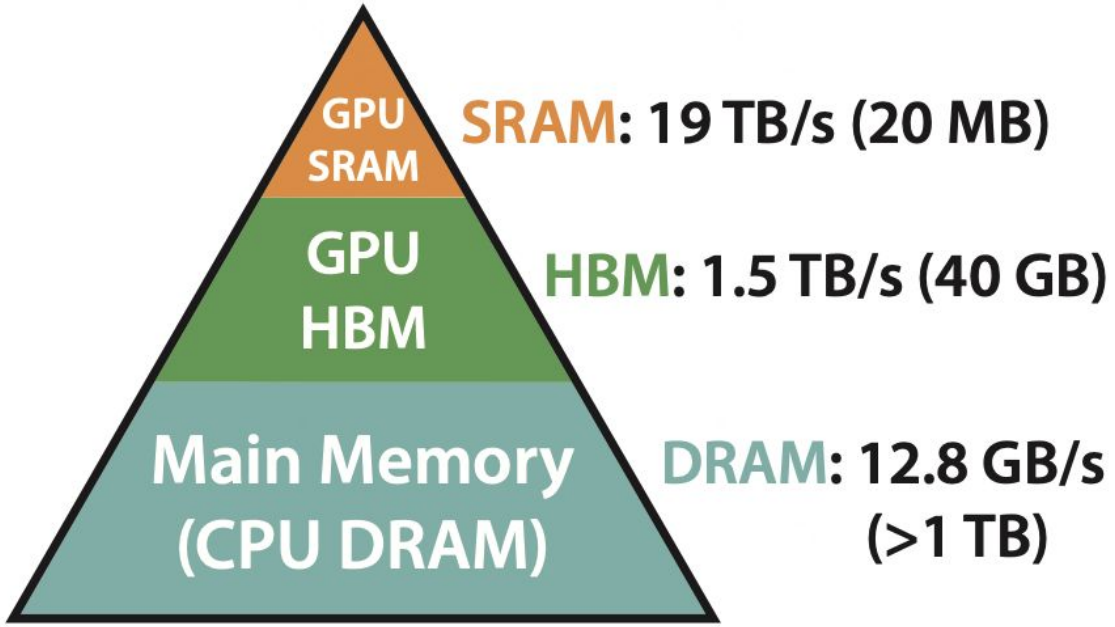
gif source

Flash Attention

Reducing memory accesses

Key Concepts

- **Static Random Access Memory.** Fastest memory with low latency, used mostly as cache memory due to its high cost and low density
- **High Bandwidth Memory.** Designed for high-bandwidth applications like GPUs and AI accelerators
- **Dynamic Random Access Memory.** The main memory used in computers, providing a balance of capacity, cost, and moderate bandwidth for general-purpose computing

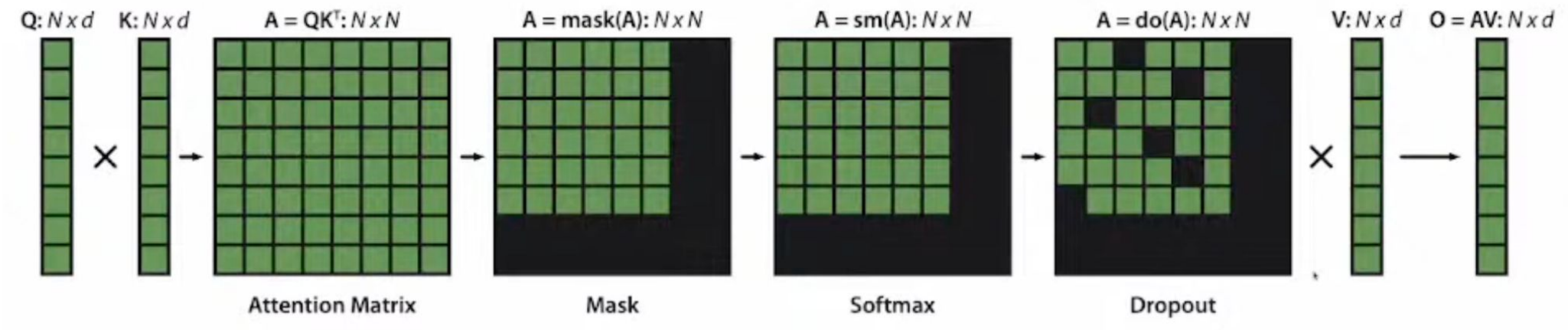


Memory Hierarchy with Bandwidth & Memory Size

2205.14135

Challenges with Standard Attention

Memory Usage. Standard attention implementation saves the matrices $A=QK^T$ and $S=\text{softmax}(A)$ in HBM, which takes $O(N^2)$ memory (often $N >> d$)



Attention is bottlenecked by memory reads/writes!

Flash Attention

How to reduce High Bandwidth Memory(HBM) reads/writes?

Challenges:

- Compute softmax reduction without access to full input
- Backward without the large attention matrix from forward

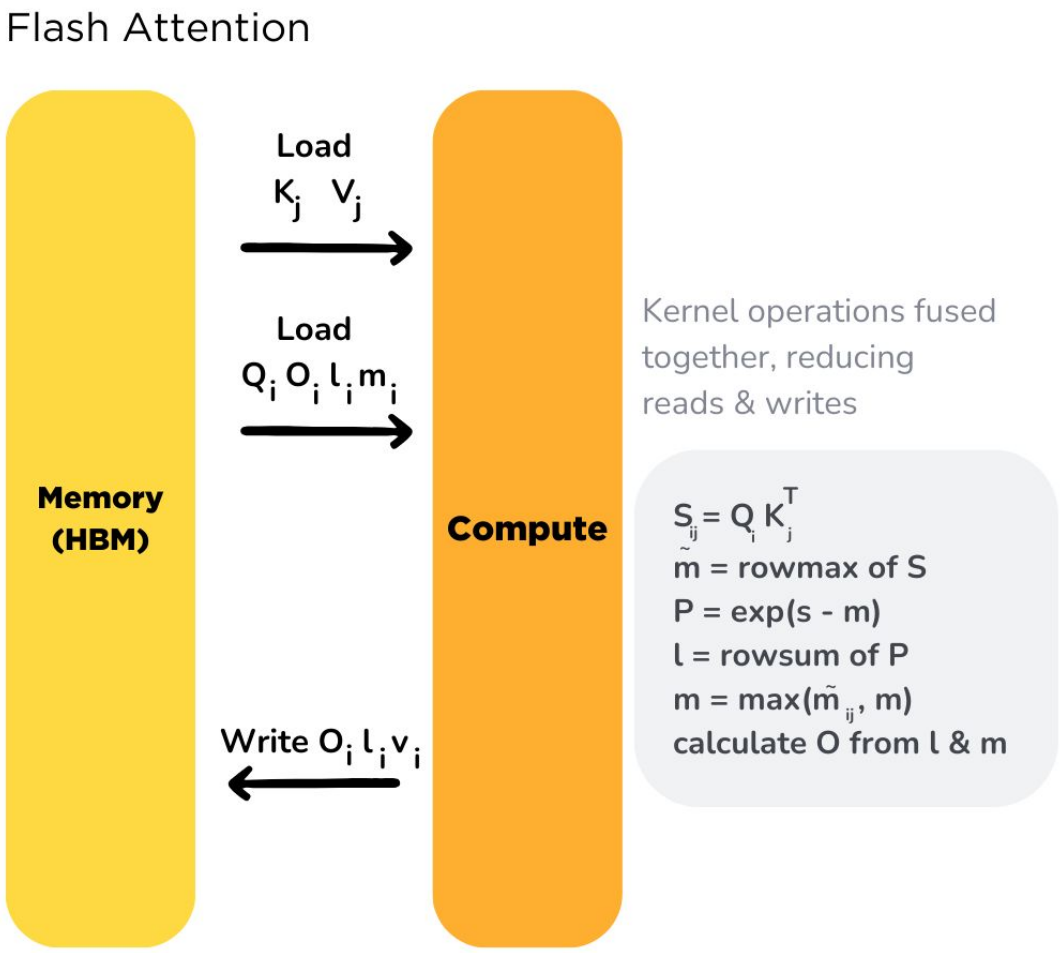
Solutions:

- **Tiling.** Restructure algorithm to load block by block from HBM to SRAM to compute attention
- **Recomputation.** Don’t store attention matrix from forward, recompute it in the backward

Flash Attention

Block-wise Computation

- Instead of computing the full attention scores for all pairs of tokens at once, FA processes the sequence in **smaller blocks**
- Each block corresponds to a subset of the sequence, and the **attention scores** are computed within and **across these blocks**



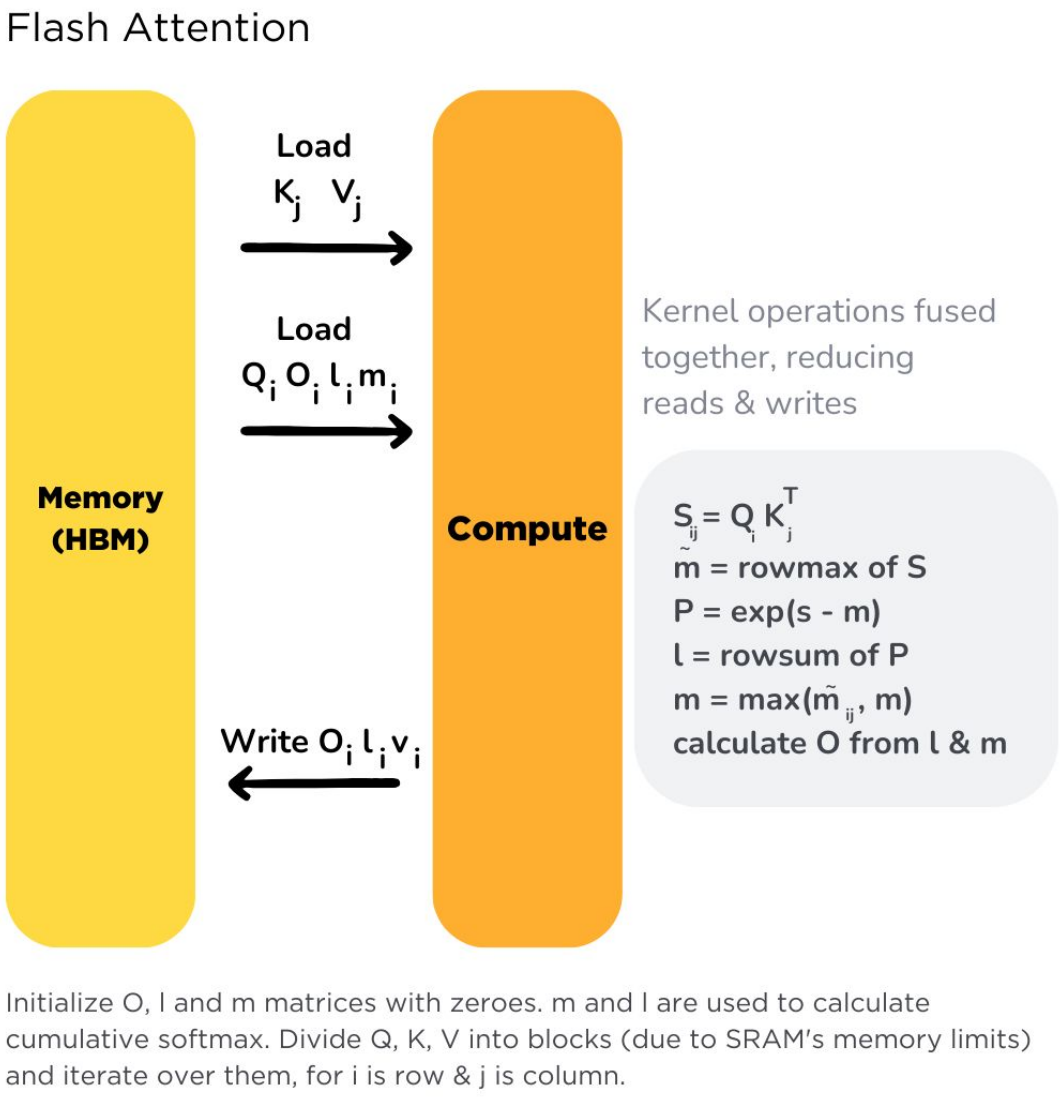
Initialize O, l and m matrices with zeroes. m and l are used to calculate cumulative softmax. Divide Q, K, V into blocks (due to SRAM's memory limits) and iterate over them, for i is row & j is column.

Hugging Face

Flash Attention

Softmax Computation

- Softmax requires normalization across the entire sequence
- FA uses a blocked softmax approach, where the **softmax is computed independently** for each block



Hugging Face

Flash Attention

Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M.

1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min\left(\lceil \frac{M}{4d} \rceil, d\right)$.
2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
3: Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \ldots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \ldots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \ldots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \ldots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \ldots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \ldots, m_{T_r}$ of size B_r each.
5: **for** $1 \le j \le T_c$ **do**
6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
7: **for** $1 \le i \le T_r$ **do**
8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1}(\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
14: **end for**
15: **end for**
16: Return $\mathbf{O}$.

2205.14135

References

KV Cache:

- https://huggingface.co/docs/transformers/main/kv_cache

Paged Attention:

- https://huggingface.co/docs/text-generation-inference/conceptual/paged_attention
- https://blog.vllm.ai/2023/06/20/vllm.html
- https://arxiv.org/pdf/2309.06180

Flash Attention:

- https://huggingface.co/docs/text-generation-inference/conceptual/flash_attention
- https://arxiv.org/pdf/2205.14135

Self-Study Materials

- https://arxiv.org/pdf/2410.03065
- https://huggingface.co/blog/kv-cache-quantization
- https://arxiv.org/pdf/2307.08691